

Vibecoding: ответы на открытые вопросы

AI-агенты и инструменты

1. Чат отвечает текстом, ты сам копируешь и выполняешь. Агент сам выполняет работу: жмёт, ходит по шагам, использует инструменты (файлы, терминал, браузер) и держит цель до конца задачи.
2. Потому что код надо ещё запустить, проверить и залить на сервер. Написанный текст с кодом — это не работающий продукт, а черновик.
3. Vibecoding — это подход, при котором ты словами описываешь, что хочешь, AI пишет код, ты смотришь результат и говоришь, что поправить, и так по кругу, пока не заработает. Ты ведёшь по смыслу (vibe), код пишет машина.
4. AI-чаты (ChatGPT, Claude) — спрашиваешь, копируешь код руками. Агенты в редакторе (Cursor, Claude Code, Windsurf, Copilot) — сами правят файлы и запускают команды. Сборщики приложений (Lovable, Bolt, v0, Replit) — описал приложение, получил готовое и сразу в сети.
5. Со сборщика приложений (Lovable или Bolt), потому что там всё под капотом и кода видеть не надо — это самый дружелюбный вход для нулевого новичка.
6. Потому что победа зависела не от умения писать код, а от того, кто лучше понимал реальную проблему (боль), которую нужно решить. AI убрал барьер написания кода.

7. Раньше эксперт знал проблему насквозь, но не мог построить решение — нанять разработчика дорого. AI дал ему возможность строить решение самому, убрав барьер кода.

Поиск и проверка идеи

1. Потому что придуманные за столом идеи (Пол Грэм зовёт их “ситком-идеи”) звучат складно, но не решают реальную боль, и юзеров под них нет.
2. Своя боль, твоя сила (то, что знаешь/умеешь лучше других), грязная работа (сложное, что все обходят стороной), жизнь в будущем (видишь то, чего ещё нет), и правило “сначала боль, потом AI” — не бери технологию в поиске применения.
3. Потому что если бы идея явно выглядела отличной, её бы уже кто-то делал. Значит настоящая хорошая идея должна казаться немного безумной со стороны.
4. Потому что узкая группа, которой продукт нужен отчаянно, даёт сильный сигнал и лояльных пользователей. Лучше 100 человек, которые обожают, чем 10 000 равнодушных.
5. Можешь ли назвать конкретных людей, кому нужна даже кривая версия 1? Ты сам этого хочешь? Это стало возможно только недавно? У тебя есть реальный новый взгляд на проблему?
6. Потому что близкие хвалят из вежливости, чтобы не обидеть, а AI обучен через RLHF быть приятным и соглашаться (sycophancy) — это его дефолтное поведение, а не оценка идеи.
7. Промпт вида: “Act as a skeptical investor. Challenge my idea, list the 5 biggest risks and why you’d pass” — то есть прямо приказать AI спорить и искать риски, а не хвалить.
8. Говори про их жизнь, а не про твою идею; спрашивай про конкретное прошлое, а не гипотетическое будущее; меньше говори, больше слушай.
9. Плохой вопрос спрашивает мнение о будущем (“нравится ли идея”, “купил бы”, “заплатил бы \$10”) — люди вежливо врут. Хороший вопрос спрашивает про факты прошлого (“как ты решаешь это сейчас”, “когда последний раз столкнулся”, “сколько это уже тебе стоит”).

10. Лендинг со списком ожидания (собери почты), предпродажа (человек платит вперёд — сильнейший сигнал), и метод “делай то, что не масштабируется” — находи первых пользователей руками.
11. Построил то, что никому не нужно (no market need) — примерно 4 из 10 провалов стартапов по этой причине.
12. Сначала “The Mom Test” (как говорить с клиентами и не получать вежливую ложь), потом “The Lean Startup” (MVP, цикл собери-измерь-научись), и “Zero to One” (создавай новое, а не копируй) — позже.

Риски и грабли vibecoding

1. Боты сканируют GitHub и находят открытые ключи за минуты, после чего злоумышленники начинают тратить деньги за твой счёт — счета могут доходить до тысяч долларов.
2. Секреты должны храниться в файле `.env`, который никогда не публикуется (добавлен в `.gitignore`). В коде остаётся только ссылка на переменную окружения, а не сам ключ.
3. Лимит трат — это страховка: если ключ утечёт или произойдёт ошибка/атака, расходы автоматически остановятся на заданном потолке, а не разорят тебя.
4. Могут появиться дыры безопасности (открытая база без пароля, отсутствие лимитов), которые AI не добавил, потому что его об этом не просили — AI делает то, что попросили, а не то, что безопасно по умолчанию.
5. Оверинжиниринг — делать сложнее, чем требует задача. Это тратит недели вместо вечера, увеличивает число мест, где может что-то сломаться, и выжигает мотивацию, из-за чего проект часто бросают.
6. Опиши задачу одним предложением (какую боль решаем), сделай минимальную рабочую версию, покажи людям и собери реакцию, усложняй только то, что реально просят.
7. Vibecoding силён для MVP, прототипов, лендингов, внутренних инструментов и простых CRUD-приложений. Опасен для платежей, медицины/финансов/регуляторики, высоких нагрузок и большого продакшен-кода.

8. AI быстро выдаёт первые ~70% решения (каркас, очевидные куски), но последние ~30% (крайние случаи, отладка, безопасность, продакшен) требуют столько же усилий, сколько всегда — “выглядит готово” ≠ “готово к продакшену”.
9. Подписка на инструмент (обычно \$10-50/мес), плюс расход кредитов/токенов, который растёт с числом итераций и циклов отладки.
10. Потому что агент отправляет весь контекст заново на каждом шаге (читает файлы, запускает команды, анализирует ошибки), в отличие от чата, где отправляется только сообщение.

Промптинг

1. Роль (кем быть AI), задача (что сделать), контекст (проект, стек, ограничения), пример (few-shot), формат ответа, и ограничения (чего избегать).
2. Плохой промпт оставляет AI всё додумывать самому (фреймворк, поля, стиль) и даёт что-то не то. Хороший промпт содержит роль, точные детали задачи, контекст стека и конкретные ограничения — и даёт именно то, что представлял автор.
3. Быть конкретным вместо размытых просьб; делить большую фичу на маленькие промпты; просить AI “думать по шагам” (chain-of-thought); давать пример (few-shot); возвращать текст ошибки вместе с кодом; итерировать от простой рабочей версии; говорить, что делать, а не чего не делать.
4. Нужно вставить и текст ошибки (например, точный текст исключения и номер строки), и сам код — тогда AI видит полную картину и может исправить причину, а не гадать.
5. Нужно остановиться, вернуться к последней рабочей версии, открыть свежий чат, разбить задачу на более мелкие шаги или сменить модель/инструмент.
6. Потому что дальнейшее повторение одного и того же промпта не меняет контекст, в котором застрял AI, — это только тратит деньги и нервы без результата.

7. Потому что новый контекст заставляет AI заново перечитать код и подойти к задаче иначе, вместо того чтобы застревать в старой, уже испорченной логике рассуждений.

Базовые технические знания

1. Потому что база — это твои рычаги контроля: без неё нельзя откатить плохое изменение, залить приложение в интернет или понять, что именно сломалось. Понимать процесс — не то же самое, что печатать код руками.
2. Git — система точек сохранения для всего проекта. Она нужна, чтобы можно было откатиться на любой момент, если AI что-то сломал, как в сейвах видеоигры.
3. Чтобы всегда была точка возврата: если следующее изменение AI сломает проект, можно откатиться к последней рабочей версии, а не терять всю проделанную работу.
4. Потому что без тестов каждое изменение — это лотерея: можно тихо сломать что-то в другом месте, чего “вроде работает” никогда не поймает.
5. Агент пишет код, сам запускает тесты, видит, что упало, чинит код и снова запускает тесты — цикл повторяется, пока все тесты не станут зелёными.
6. Не со ста тестов, а с одного — на самый важный сценарий (например, логин, оплата, отправка формы), дальше добавлять по мере роста приложения.
7. Docker-контейнер — это коробка, в которую упаковано приложение вместе со всем, что ему нужно (код, библиотеки, настройки), и она работает одинаково в любом окружении.
8. Docker решает проблему “у меня на компе работало, а у вас нет” — контейнер ведёт себя одинаково у автора, у друга и на сервере.
9. Домен — это человеческий адрес сайта (например, myapp.com), который арендуется. Хостинг — это всегда включённый компьютер, где физически живёт приложение.

10. Задеплоить — значит залить приложение на сервер/хостинг, чтобы оно стало доступно людям в интернете, а не только на локальном компьютере автора.

Хранение кода и проекта

1. Код должен храниться в Git-репозитории, а копия — на удалённом сервисе вроде GitHub, чтобы был и локальный, и облачный бэкап с полной историей изменений.
2. GitHub даёт облачную копию (бэкап) проекта и позволяет продолжить работу или восстановить проект даже при потере локального компьютера, плюс упрощает совместную работу.
3. Ветки позволяют делать безопасную копию проекта для экспериментов, не ломая основную рабочую версию, — если эксперимент не удался, просто не сливаешь ветку в основную.

Прокачка AI и повторяющиеся инструкции

1. Записать инструкции один раз в специальный файл (CLAUDE.md, AGENTS.md или SKILL.md), который AI читает автоматически в начале сессии или по необходимости, вместо повторения промпта вручную.
2. CLAUDE.md — правила проекта, читаются в начале каждой сессии (устав сотрудника). AGENTS.md — общий стандарт для разных инструментов. SKILL.md — навык под конкретную задачу, загружается по необходимости.
3. MCP (Model Context Protocol) — единый стандарт подключения AI к твоим инструментам и данным (как USB-C для AI), чтобы AI работал с реальными заметками, тикетами и кодом, а не с пустого листа.
4. Obsidian, Jira, GitHub, Google Drive, Slack, база данных — и другие сервисы, к которым можно подключить AI через готовые коннекторы.

Как приходят идеи и как их формулировать для AI

1. Аналитик может задавать AI чёткое ТЗ через привычные артефакты: BR (зачем вообще делаем), SRS с FR (что система умеет) и NFR (как быстро, надёжно, безопасно) — это и есть контроль над тем, что построит AI.
2. Spec-driven development — разработка через спецификацию: размытая просьба даёт размытый результат, а чёткие FR и NFR дают AI ясное понимание, что строить и с какими ограничениями — то есть спецификация — это контроль над AI.
3. Достаточно словами описать структуру системы или процесс, и AI сгенерирует UML-диаграмму (классы, связи) или BPMN-схему (шаги процесса), которую затем нужно проверить и поправить.

Дисциплина и правила

1. Коммить часто, читать код AI как черновик (а не истину), двигаться малыми шагами, самому решать вопросы денег и безопасности, не править продакшен напрямую (держат бэкапы), и знать, когда звать живого разработчика.
2. Потому что AI выбирает решение, которое “работает”, а не обязательно то, которое безопасно — вопросы денег и безопасности требуют человеческого решения и ответственности.
3. Когда дело касается реальных денег, данных людей, или когда сам не понимаешь, что происходит в системе — в таких случаях риск ошибки слишком высок для чистого vibecoding.